

Mellow Language Reference

Core syntax, keywords, and built-in functions

Mellow Language project

1. 1. Overview

Mellow is an English-influenced programming language implemented in C. Function calls use angle brackets, code blocks use square brackets, and curly braces are used for array data. Arithmetic and comparison are expressed with named built-in functions rather than operators.

This reference describes the current executable core. Keywords marked "reserved" are recognized by the lexer but are not yet executable in the interpreter.

2. 2. Basic Syntax

Function calls:

```
print<"Hello, Mellow!">
add<2, 3>
print<add<2, mul<3, 4>>>
```

Blocks:

```
if gt<score, 10> [
    print<"high score">
]
```

Comments use `//` for one line and `/* ... */` for multiple lines. Newlines may be used between arguments and delimiters in calls and collection literals.

3. `contains` is an alias for `.B inside`. `null` The absence of a value. The literal is `.B null`. `number abs<a>` Common numeric helpers. `.B min` and `.B max` select a number; `.B clamp` limits a value to a range. `.B abs` returns magnitude. `values`. Examples: `.B 42` and `.B 3.14159`. `string is_null<a>` The related predicates `.B is_number`, `.B is_string`, `.B is_array`, and `.B is_list` Text enclosed in double quotes. Example: `.B "hello"`. `char` A single character enclosed in single quotes. Character values currently use the same string representation at runtime. `bool` The boolean literals `.B true` and `.B false`. `array` An ordered collection written with curly braces. Example: `.B {1, 2, 3}`. `list` An ordered dynamic collection created with `.B list<...>`. Example: `list<"red", "green", "blue">`.

`let` defines a mutable variable:

```
let name = "Mellow"
set<name, "Mellow Language">
```

`const` defines a variable that cannot be reassigned or passed as the first argument to `.B set`:

```
const PI = 3.14159
```

4. **4. Implemented Keywords** `true` Creates the boolean value `true`. `false` Creates the boolean value `false`. `null` Creates the null value. `let` Declares a mutable variable using `.B let name = expression`. `const` Declares an immutable variable using `.B const name = expression`. `if` Executes a block when an expression is truthy. Truthy values are `true`, nonzero numbers, nonempty strings, and nonempty collections.

```
if eq<name, "Mellow"> [  
  print<"matched">  
]
```

else Provides the alternative block for .B if. Chained .B else if syntax is supported. **while** Repeats a block while its condition is truthy.

```
while gt<count, 0> [  
  print<count>  
  set<count, dec<count>>  
]
```

loop Repeats a block indefinitely until .B break or a runtime error occurs. **break** Leaves the nearest loop. **continue** Skips to the next iteration of the nearest loop. **return** Returns a value from a named function. With no expression it returns null. **func** Defines a named function with parameters and a square-bracket body.

```
func square<x> [  
  return mul<x, x>  
]  
print<square<5>>
```

raise Raises a runtime exception and exits the current program with a nonzero status. The usual form is .B raise<"message">.

5. 5. Arithmetic Builtins

Arithmetic operators such as +, -, *, /, and % do not exist in Mellow. **add<a,b>** Returns the numeric sum. It also concatenates two strings or two collections of the same kind. **sub<a,b>** Returns a minus b for numbers. **mul<a,b>** Returns a times b for numbers. **div<a,b>** Returns a divided by b. Division by zero is a runtime error. **mod<a,b>** Returns the floating-point remainder of a divided by b. **pow<a,b>** Returns a raised to the power b. **sqr<a>** Returns the square root of a. **inc<a>** Returns a plus one. **dec<a>** Returns a minus one.

6. 6. Comparison and Logic Builtins

Comparison operators such as ==, !=, <, >, <=, and >= do not exist as operators. The following functions return booleans. **eq<a,b>** True when the values are equal, including recursive collection equality. **neq<a,b>** True when the values are not equal. **lt<a,b>** True when numeric a is less than numeric b. **gt<a,b>** True when numeric a is greater than numeric b. **lte<a,b>** True when numeric a is less than or equal to numeric b. **gte<a,b>** True when numeric a is greater than or equal to numeric b. **equal<a,b>** Readable alias for .B eq<a, b>. **not_equal<a,b>** Readable alias for .B neq<a, b>. **less<a,b>** Readable alias for .B lt<a, b>. **greater<a,b>** Readable alias for .B gt<a, b>. **less_equal<a,b>** Readable alias for .B lte<a, b>. **greater_equal<a,b>** Readable alias for .B gte<a, b>. **and<a,b>** Returns the boolean conjunction of a and b. **or<a,b>** Returns the boolean disjunction of a and b. **not<a>** Returns the boolean negation of a. **inside<value,container>** Returns only a boolean. For strings, it checks whether the first string occurs inside the second. For arrays and lists, it checks whether an equal item exists. **contains** is an alias for **inside**.

7. 7. Collection and Conversion Builtins **list<items>** Creates a list from zero or more values. **len<value>** Returns the character count of a string or the item count of an array/list. **type<value>** Returns a string naming the runtime type: null, number, string, bool, array, or list. **to_string<value>** Converts primitive values to text. Collections currently return a collection placeholder string and are displayed directly by .B print. **print<values>** Prints each argument separated by a space, followed by a newline. **set<name,value>** Replaces the value of a mutable variable. The first argument is a variable name, not an evaluated expression. **get<container, key>**, **has<container, key>**, **put<dictionary, key, value>** Read a dictionary key or collection index, test whether it exists, or add and replace a dictionary entry. **step>range<start,end>** Creates a numeric list with an exclusive end. The step is optional and defaults to one. **for item in collection [...]** Iterates over an array or list. **input<"prompt">**input<>or Reads one terminal line as a string. An optional string prompt is printed before reading. **to_int<value>****to_number<value>**,**to_float<value>**, Convert numeric text. .B **to_int** truncates the numeric value. **to_bool<value>** Converts booleans, nonzero numbers, or the strings true, false, 1, and 0.

`input<"prompt">` `input<>` or Reads one terminal line as a string. An optional string prompt is printed before reading. `to_int<value>` `to_number<value>`, `to_float<value>`, Convert numeric text. `.B to_int` truncates the numeric value. `to_bool<value>` Converts booleans, nonzero numbers, or the strings `true`, `false`, `1`, and `0`. `abs<a>`, `min<a, b>`, `max<a, b>`, `clamp<a, low, high>` Common numeric helpers. `abs` returns magnitude; `min` and `max` select a number; `clamp` limits a value to a range. `round<a>` `floor<a>`, `ceil<a>`, Return the corresponding rounded numeric value. `is_null<a>`, `is_number<a>`, `is_string<a>`, `is_array<a>`, `is_list<a>` Return a boolean indicating whether a value has the requested runtime type.

8. 9. Modules and Imports

An import executes declarations from another Mellow file in the same runtime. Use an angle-bracket selector to import only named functions or classes; variables and top-level executable statements are ignored. Paths are relative to the process working directory and may name files in subfolders.

```
import "test/lib/greetings.mll" <welcome>
print<welcome<"Mellow">>
```

The legacy form `.B import "path.mll" <>` imports every function and class. Imported files may import other files. Import cycles are not yet detected.

9. 10. Private Methods

Inside a class, `.B private func` declares a private method. It may be called by methods of its declaring class or subclasses, including through `.B this.method<>` or `.B super.method<>`. External callers and unrelated classes are rejected. Nested classes do not automatically receive private access.

10. 8. Interpolation

Expressions inside a string using `{name}` are replaced with the current value of the named variable:

```
let user = "Ada"
print<"Hello, {user}!">
```

Interpolation currently supports variables containing strings, numbers, and booleans. Unknown names remain unchanged.

11. 9. Functions and Recursion

Named functions support parameters, local variables, return values, and recursion. Arguments are evaluated before the function is called.

```
func factorial<n> [
  if lte<n, 1> [
    return 1
  ]
  return mul<n, factorial<dec<n>>>
]
print<factorial<5>>>
```

12. 10. Object-Oriented Programming

Classes use square-bracket bodies. Fields use `.B let`, constructors use the method name `.B init`, and methods use angle-bracket calls.

```
class Person [  
  let name = ""  
  func init<n> [  
    set<this.name, n>  
  ]  
  func greet<> [  
    print<"Hello, ", this.name>  
  ]  
]  
let person = Person<"Ada">  
person.greet<>
```

Single inheritance uses `.B class Student : Person [...]`. Inherited fields, dynamic method lookup, overriding, and parent calls through `.B super.method<>` are supported. `.B destroy<instance>` calls an optional `.B free<>` method before releasing the instance.

Visibility modifiers, static members, automatic garbage collection, and structured access control are reserved for later runtime layers.

13. 11. Command-Line Tools make Builds the interpreter. Run it from the project root. `./mellowfile.mll` Executes a Mellow source file. `file.mll/mellow--tokens` Prints the lexer token stream with source locations. `./mellow--repl` Starts the interactive read-evaluate-print loop. Type `.B exit` to leave it.

14. 12. Reserved Keywords

The following words are lexed and reserved for the next runtime layers: **for**, **public**, **private**, **import**, **try**, **catch**, **fn**, and.

for is reserved for collection iteration. `.B public` and `.B private` are reserved for access control. `.B import` is reserved for modules. `.B try` and `.B catch` are reserved for structured error recovery. `.B fn` is reserved for anonymous functions and closures. These features are specified in the project roadmap but are not executable in the current core.

15. 13. Current Limitations

Dictionaries, indexing, default parameters, lambdas, closures, structured try/catch recovery, modules, visibility, static members, and garbage collection are not yet implemented. The lexer recognizes much of their vocabulary so these features can be added without changing the central call and block syntax.

